

BUG REGRESSIVO

PREFAZIONE

Salve, questo documento è di estrema importanza, e desidero spiegare il motivo.

Il presente documento è stato redatto da me, ma la semplificazione dei contenuti è stata discussa e sviluppata in collaborazione con il collega Simone Lacroce, il quale, pur avendo già superato l'esame, si è impegnato attivamente nel rendere l'argomento più accessibile, con l'unico obiettivo di ridurre la complessità del concetto e renderlo pienamente comprensibile a chiunque.

L'argomento che sto per introdurre è fondamentale ai fini dell'esame, e non può in alcun modo essere tralasciato.

Si tratta di un tema piuttosto concettuale e creativo, privo di qualunque procedura algoritmica.

Sto parlando del bug regressivo.

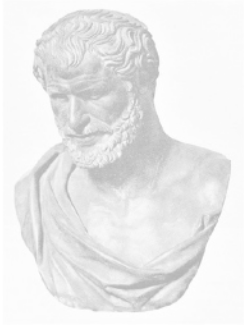
Raccomando vivamente la lettura di questo documento, poiché contiene tutte le informazioni necessarie per affrontare con successo la prova scritta del Prof. Falessi.

L'esempio e la relativa riadattazione sono a cura di Simone Lacroce, il cui elaborato progettuale e prova d'esame sono stati entrambi valutati con il massimo dei voti (30/30).

Il bug regressivo non è un mostro informatico pronto a confonderci, ma solo un concetto che ha bisogno di essere raccontato nel modo giusto.

Il bug regressivo non è un enigma irrisolvibile, ma un invito a riflettere su un concetto antico quanto la storia stessa: l'errore che ritorna.

BUG REGRESSIVO - UN VIAGGIO



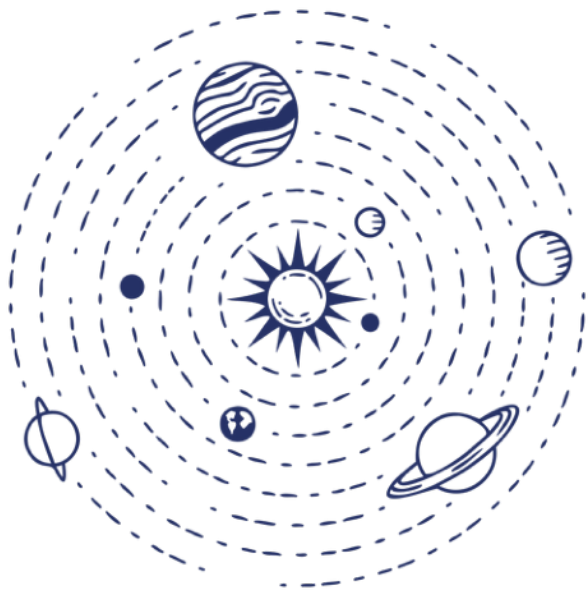
Come direbbe *Heraclito*, “nulla si mantiene immutato, tutto scorre”.

E il software, come la vita, cambia costantemente: nuove versioni e aggiornamenti. Ma in ogni cambiamento si nasconde un rischio. Un piccolo passo avanti nel codice può riportarci a un passo indietro nella funzionalità.

PIÙ MODIFICHIAMO, PIÙ RISCHIAMO DI ROMPERE CIÒ CHE FUNZIONAVA.

Allo stesso modo, l'ingegneria del software progredisce correggendo i propri errori, ma ogni correzione è una nuova ipotesi da testare.





Immaginate per un momento che il nostro software sia un sistema solare. Ogni classe è un **planeta**, ogni funzione una luna, e il codice che li tiene insieme è la forza di gravità che regola l'equilibrio del tutto. Ogni orbita, ogni rotazione, è precisa, armonica, perfetta.

Ora, come farebbe un ingegnere curioso o un astrofisico ambizioso, decidiamo di “migliorare” l'universo.

Aggiungiamo un nuovo pianeta, magari più luminoso, più bello, più efficiente.

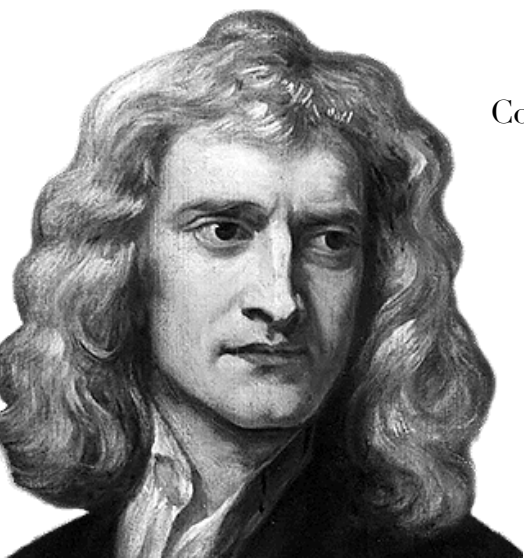
Ma in questo gesto di evoluzione, alteriamo impercettibilmente la gravità.

I pianeti vicini iniziano a muoversi diversamente, le orbite si deformano, e ciò che prima era perfettamente bilanciato comincia a collassare su sé stesso.



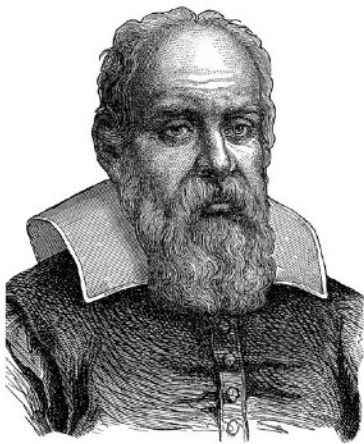
È proprio lì che nasce il bug regressivo: non nel nuovo pianeta, ma nella reazione a catena invisibile che il cambiamento ha generato nell'intero sistema.

Un equilibrio cosmico infranto da una modifica apparentemente innocua.



Come direbbe **Isaac Newton**, “*Ogni azione ha una reazione uguale e contraria*”.

Perciò, la modifica può generare una reazione indiretta in un'altra parte del sistema, che magari non era stata toccata. Quella reazione, a volte invisibile e contraria alle aspettative, è proprio ciò che noi chiamiamo bug regressivo.



E forse, come direbbe **Galileo**, “*La natura è scritta in linguaggio matematico*”.

Ma la vera maestria sta nel saper preservare l’armonia tra le formule già esistenti.

FILOSOFIA DELL’ECO UMANO

Ogni scelta che facciamo nella vita, come in un software, genera conseguenze che non sempre possiamo controllare. Cerchiamo di migliorare qualcosa e, senza volerlo, feriamo ciò che funzionava già. Il bug regressivo è lo specchio di questa condizione: una metafora della fragilità della creazione, dove anche la mano più esperta deve imparare a essere umile di fronte alla complessità.

Il bug regressivo ci ricorda che il progresso non è mai unidirezionale.

Ogni passo avanti nella complessità porta con sé **un’ombra di regressione**, un debito tecnico o logico da estinguere.

È una metafora del nostro rapporto con la tecnologia: più la perfezioniamo, più dobbiamo vigilare per non diventarne vittime inconsapevoli.

Io, mi sento anche di asserire la seguente frase:

“Ogni volta che correggiamo il nostro codice, è il codice che, silenziosamente, corregge noi.”

È una di quelle verità che sembrano un botto paradossali: nel tentativo di rendere il software più stabile, impariamo quanto siamo fragili; nel tentativo di eliminare gli errori, scopriamo che siamo noi la fonte del disordine.



Fermatevi un momento e osservatevi: guardatevi allo specchio, nel riflesso di una vetrina o nel finestrino di un’auto. Il software è l’unico specchio che riflette non il nostro volto, ma la nostra logica.

IL PENSIERO UMANO HA I SUOI LIMITI

Quando progettiamo un sistema, stiamo facendo qualcosa di molto simile a ciò che fa la mente umana ogni giorno: analizziamo e costruiamo modelli.

Ogni variabile che dichiariamo, ogni funzione che scriviamo, ogni errore che correggiamo è una manifestazione concreta del nostro modo di ragionare.

Ogni azione, anche la più piccola, ha conseguenze. Ogni miglioramento può generare una nuova fragilità.

Secondo me bisogna riconoscere che dietro ogni codice c'è un pensiero umano, con i suoi limiti, le sue contraddizioni, e la sua creatività. In fondo, il software è l'immagine digitale della mente che lo ha creato.

E lo dico da appassionato e da semplice studente.

FINE PREMESSA - ORA SI COMINCIA.

ESEMPIO A NAPOLI



C'è un esempio ormai celebre, e anche piuttosto ironico, che spiega perfettamente cosa succede quando l'integrazione tra componenti fallisce: il caso dei treni acquistati per la metropolitana di Napoli che non entravano nelle gallerie o nelle banchine delle stazioni. Tutti i treni erano perfettamente funzionanti. I progettisti li avevano costruiti con cura, rispettando le specifiche tecniche.

Anche le stazioni, da parte loro, erano operative e realizzate secondo i piani.

Il problema non era quindi nei singoli moduli, ma nel loro incontro: quando il sistema è stato assemblato, cioè quando treno e infrastruttura sono stati messi insieme, si è scoperto che le dimensioni non erano compatibili.

Ecco il cuore dell'integration testing gone wrong: ogni parte del sistema è corretta da sola, ma l'integrazione fallisce perché nessuno ha verificato se le due parti comunicano o si adattano correttamente.

ESEMPIO ALLE MACCHINETTE



Immaginiamo un distributore automatico perfettamente funzionante.

Il software che gestisce il pagamento funziona, il sistema che riconosce il credito è corretto, i sensori che rilevano i prodotti sono calibrati. E anche la bottiglia è perfettamente prodotta, con dimensioni regolari e tappo sigillato. Eppure, nel momento in cui i due sistemi, il distributore e la bottiglia, devono interagire, qualcosa va storto.

La bottiglia rimane incastrata, bloccando tutto il meccanismo.

Non è un guasto meccanico, e nemmeno un errore di progettazione isolato: è un classico problema di integrazione.

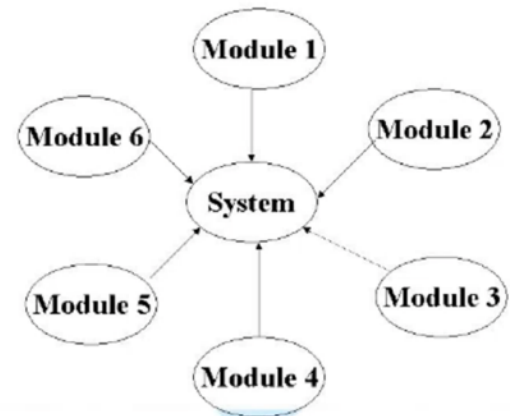
Significa che i componenti sono corretti singolarmente, ma non sono stati testati insieme in condizioni reali.

INTEGRAZIONE - BIG BANG

Una volta testati singolarmente i vari moduli, essi devono essere integrati tra loro.

Il primo approccio, chiamato Big Bang, prende il nome proprio dall'idea di un'esplosione caotica: consiste infatti nel integrare tutti i moduli contemporaneamente e testare il sistema nel suo complesso.

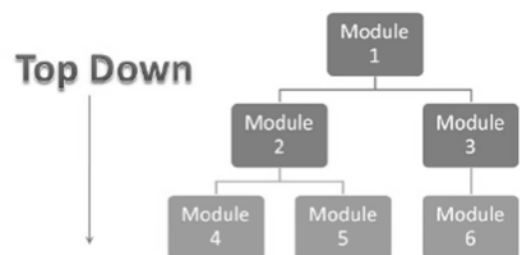
Questo metodo ha però un notevole svantaggio: quando si verificano errori, **risulta estremamente difficile individuarne la causa e correggerli**, poiché tutti i componenti vengono inseriti nello stesso momento.



INTEGRAZIONE - TOP DOWN

L'integrazione e il testing avvengono dall'alto verso il basso, seguendo la struttura gerarchica o il flusso di controllo del sistema. In pratica, si parte dai moduli di livello più alto, ad esempio l'interfaccia utente o il menu principale, e si integrano progressivamente i moduli di livello inferiore (DALLA GUI ALLA LOGICA).

I componenti che non sono ancora stati sviluppati vengono sostituiti da **stub**, cioè piccole porzioni di codice che simulano in modo semplificato il comportamento dei moduli mancanti (DUMMIES).



Tuttavia, il principale svantaggio è che le funzionalità di base, cioè quelle dei moduli più bassi nella gerarchia, vengono testate solo nelle fasi finali, rendendo più difficile individuare precocemente eventuali difetti fondamentali.

ESEMPIO DI STUB:

```
public class StubPagamento {  
    public boolean eseguiPagamento(double importo) {  
        System.out.println("Pagamento simulato di " + importo + "€");  
        return true; // risposta dummy=stupida  
    }  
}
```

Questo è uno stub del pagamento, cioè una funzione fittizia che, invece di elaborare realmente il pagamento, restituisce sempre un valore fisso come “OK” o “Transazione riuscita”.

Uno stub è un pezzo di codice “fittizio” (**dummy**) che imita il comportamento di un modulo reale non ancora disponibile.

Serve per permettere al modulo superiore di essere testato come se il modulo inferiore esistesse già.



Successivamente vado a implementare realmente i miei metodi, piano piano.

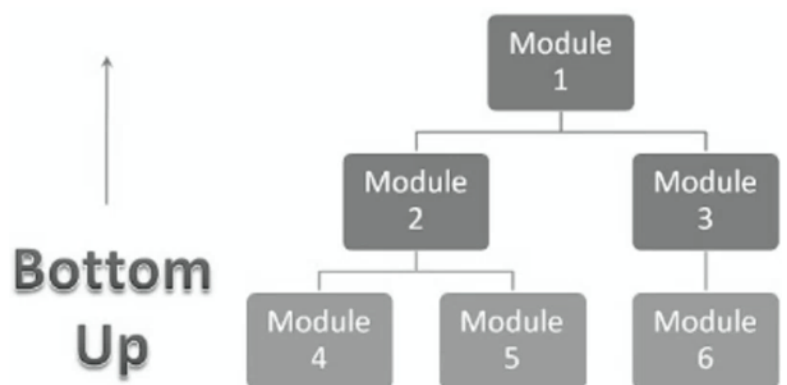
INTEGRAZIONE - BOTTOM UP

L'approccio Bottom Up è l'esatto
opposto del Top Down:

in questo caso, l'integrazione e il
testing avvengono dal basso verso l'alto,
seguendo la gerarchia del sistema. Si
parte cioè dai moduli di livello più
basso, quelli che svolgono le funzioni
più elementari (come la gestione dei
dati o delle connessioni), per poi
integrare progressivamente i moduli di

livello superiore, fino a raggiungere quelli più complessi e visibili all'utente.

Quando i moduli di livello superiore non sono ancora stati sviluppati, vengono
sostituiti da **driver**, ovvero programmi di test dummy che simulano il comportamento
dei componenti chiamanti.



ESEMPIO

Immagina di sviluppare un'applicazione che gestisce ordini online.

Hai già creato i moduli più “bassi”, ad esempio quelli che si occupano di salvare un ordine nel database o calcolare il prezzo totale, ma non hai ancora implementato la parte grafica o la logica di gestione dell'interfaccia. In questo caso puoi scrivere un driver che chiama i metodi del modulo “Ordine” passando dei dati di prova.

Il driver serve quindi come programma temporaneo di test, che ti consente di verificare se i moduli sottostanti funzionano correttamente prima di integrarli in un sistema completo.

```
public class DriverOrdine {  
    public static void main(String[] args) {  
        ModuloOrdine ordine = new ModuloOrdine();  
        ordine.aggiungiProdotto("Mouse", 2, 19.90);  
        ordine.aggiungiProdotto("Tastiera", 1, 39.90);  
        System.out.println("Totale ordine: " + ordine.calcolaTotale());  
    }  
}
```

DIFFERENZA TRA DRIVER E STUB - DEFINIZIONE FALESSI

Stub: uso vero di una logica implementata in maniera finta.

Driver: uso finto di una logica implementata in maniera vera.

TESTING DI SISTEMA

Il System Testing viene condotto dal punto di vista dell'utente finale.

Non serve conoscere la struttura interna del codice o l'architettura del sistema, non si tratta di un test “tecnico” ma funzionale, che mira a verificare se il sistema fa ciò che dovrebbe fare, in base ai requisiti. Per questo motivo, si parla anche di **black-box testing**: il tester osserva solo gli input e gli output, senza interessarsi al funzionamento interno.

Verifica che il sistema rispetti tutti i requisiti funzionali e non funzionali (prestazioni, sicurezza, compatibilità, ecc.);

LOAD TESTING

È un test di tipo **Black Box**: quindi il tester **non guarda dentro il codice** (come avverrebbe in un test White Box), ma **valuta gli input e gli output del sistema**, misurando prestazioni, tempi di risposta e stabilità.

Il **Load Testing** è una tecnica di **Performance Testing** che valuta la capacità del sistema di funzionare correttamente sotto diversi livelli di carico, sia in condizioni **normali** che di **picco**.

L'obiettivo del load testing è valutare il comportamento del sistema sia in condizioni normali che di picco, misurando parametri come:

- il tempo di risposta,
- la stabilità,
- e la capacità complessiva di gestire più richieste senza rallentamenti o blocchi.

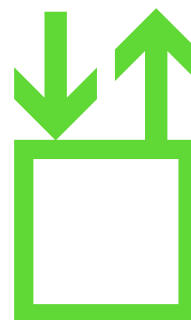
Serve quindi a determinare **fino a che punto il sistema può sostenere il carico prima che le prestazioni inizino a degradarsi**.

ESEMPIO: Immagina un sito di biglietteria online: in condizioni normali, gestisce 1.000 utenti al giorno senza problemi. Ma durante il concerto di Lady Gaga, decine di migliaia di persone si collegano nello stesso momento. Il load testing simula proprio questo scenario, verificando se il sistema continua a rispondere correttamente.

Concetto di soglia nei sistemi informatici

È importante ricordare che **tutti i sistemi informatici hanno una soglia oltre la quale non riescono più a funzionare correttamente**.

Questa soglia rappresenta il limite massimo di carico che il sistema è in grado di sostenere prima che le prestazioni degradino o che si verifichi un blocco totale. Un esempio intuitivo è quello dell'**ascensore**: ogni ascensore ha una **capienza massima**, stabilita in base al suo progetto e alla potenza del motore.



Finché il numero di persone o il peso totale rimangono al di sotto di quella soglia, il sistema funziona perfettamente.

LOOP TESTING

Il **Loop Testing** è una tecnica di test che si concentra sui **cicli (loop)** presenti nel codice, con l'obiettivo di verificare che funzionino correttamente in tutte le situazioni possibili.

Questo tipo di test serve a individuare errori come:

- loop che non terminano mai (*infinite loops*),
- iterazioni eseguite un numero sbagliato di volte,
- o condizioni che impediscono al ciclo di partire.

Se il ciclo ha un limite massimo di iterazioni (n), bisogna testare anche i casi limite:

- esecuzione $n-1$ volte, appena sotto la soglia,
- esecuzione n volte, esattamente al limite,
- esecuzione $n+1$ volte, per vedere cosa accade se il limite viene superato.

E questo lo facciamo anche a nostra insaputa, tutti i giorni quando programiamo.

Il **Loop Testing** è una tecnica di **white-box testing**: il nome “*scatola bianca*” deriva dal fatto che il codice è **trasparente**, visibile come se guardassimo all'interno di una scatola di vetro: il tester può “vedere” tutto ciò che avviene dentro.

NEGATIVE TESTING

Il Negative Testing serve a verificare che il sistema non vada in errore quando riceve input imprevisti, errati o anomali.

In altre parole, si testa la capacità del software di resistere a condizioni anomale senza bloccarsi, crashare o restituire comportamenti non gestiti. Mentre il **Positive Testing** controlla che l'applicazione funzioni correttamente con dati validi, il **Negative Testing** cerca intenzionalmente di “rompere” il sistema per assicurarsi che sappia gestire gli imprevisti in modo controllato.

Il Negative Testing è un tipo di Black Box Testing: il tester non ha bisogno di conoscere come funziona il codice internamente, ma si limita a inserire input errati o imprevisti e osservare come il sistema reagisce dall'esterno.

A QUESTO PUNTO POSSIAMO INTRODURRE I CONCETTI PIÙ IMPORTANTI PER I QUALI QUESTO DOCUMENTO È NATO.

TEST REGRESSIVO

Il **test regressivo** è un tipo di test software che viene eseguito dopo una modifica al codice, per verificare che le funzionalità già esistenti continuino a funzionare correttamente: *si testa nuovamente il software a cui sono state fatte modifiche.*

Se, durante questo test regressivo trovo un bug, esso prende il nome di **bug regressivo**.

Il bug regressivo è il **problema**.

Il *test regressivo* è il metodo per scoprirlo.

Il test regressivo serve a scoprire i bug regressivi.



Bug Informatico

Molti sviluppatori fanno fatica a crederci, ma piccole modifiche in una parte del sistema possono spesso causare problemi in altre parti.

NON SI PUÒ TESTARE TUTTO.

Ogni cambiamento nel codice, anche minimo, richiederebbe teoricamente un controllo completo di tutto il sistema. Ma questo sarebbe impossibile in termini di tempo e risorse.

Inoltre, nel mondo reale i test regressivi impiegano troppo tempo, **bloccano l'intero ciclo di sviluppo**.

Gli sviluppatori devono aspettare la fine dei test per poter integrare nuove funzionalità, con conseguenti **ritardi e costi maggiori**. Per questo motivo, nei progetti reali si

selezionano solo i test più significativi o le aree più a rischio da rieseguire dopo ogni modifica.

"Bisogna capire quali particolari test, del codice già testato, vanno nuovamente eseguiti."
-D.Falessi

Non serve rieseguire un test se la parte di codice che esso verifica **non è stata toccata o influenzata** dalla modifica.

Un piccolo accenno: Il framework di test più diffuso in Java è **JUnit**.

È una libreria open source che consente di scrivere e automatizzare test unitari, cioè test che verificano il comportamento di singole unità di codice (metodi o classi). Però ne parlerò forse (perché allo stato attuale ancora non l'ho scritto) in un documento a parte.

Adesso entriamo nel cuore della discussione. Durante la prova d'esame scritta, qualora venga proposta una domanda sul bug regressivo, ciò che dobbiamo conoscere è esattamente quanto segue.

TIPI DI CAMBIAMENTI

I cambiamenti che un software può subire, e che possono dare origine a un **bug regressivo**, si suddividono in **quattro categorie principali**:

1. Modifiche **adattive**, introdotte per adeguare il sistema a nuovi ambienti o

tecnologie (ad esempio, un nuovo sistema operativo o una nuova versione di un database). Se l'OS non supporta più il mio programma, devo modificare il codice per adattare quest'ultimo al nuovo OS. Cambio il codice perché qualcosa attorno a me è cambiato (l'ambiente in particolare è cambiato).

2. Modifiche **correttive**, effettuate per risolvere errori o malfunzionamenti rilevati dagli

utenti o durante le fasi di test. Qualcuno ha trovato un errore nel codice e io cambio il mio software per eliminarlo.

3. Modifiche **perfettive**, volte a migliorare le prestazioni, l'usabilità o l'efficienza del sistema, senza alterarne la logica funzionale. Sono interventi sul software effettuati **non** per correggere errori, ma per **migliorarne le prestazioni, la qualità o l'esperienza d'uso**. Si tratta quindi di **azioni di ottimizzazione o potenziamento**, che mirano a rendere il sistema più efficiente, leggibile, veloce o gradevole, **senza alterare la logica funzionale principale**.

In altri termini, il software continua a fare le stesse cose, ma **le fa meglio**.

4. Modifiche **preventive**, realizzate per aumentare la manutenibilità o l'affidabilità del software, prevenendo possibili problemi futuri.

Si tratta di azioni **proattive**, volte a **prevenire problemi futuri**, piuttosto che a risolvere difetti già esistenti. In altre parole, con una modifica preventiva **si rinforza la struttura interna del software**, preparandolo ad affrontare con maggiore robustezza i cambiamenti futuri o l'aumento di complessità del sistema.

Esempio: **sostituzione del database** con uno più scalabile, per evitare futuri limiti di capacità o lentezze nel recupero dei dati.

ELEMENTI DI UN BUG REGRESSIVO

1. Condizione Iniziale.
2. Modifiche Introdotte.
3. Bug Introdotta dopo la modifica.
4. Prevedibilità.
5. Soluzione/Miglioramento.

All'esame si deve descrivere un esempio di bug regressivo di un tipo che viene scelto tra i 4 appena discussi.

Per questo bug bisogna specificare 5 condizioni.

Condizione iniziale:

Descrive la situazione di partenza, ossia come il sistema funzionava correttamente prima della modifica.

Modifica introdotta:

Indica il tipo di cambiamento apportato (adattivo, correttivo, perfettivo o preventivo) e il suo scopo.

Bug regressivo introdotto dopo la modifica:

Descrive il malfunzionamento che si è manifestato dopo la modifica, specificando che l'errore **non è direttamente nella modifica, ma in una parte del sistema che prima funzionava.**

Prevedibilità:

Analizza quanto fosse **ragionevole o prevedibile** che il bug potesse verificarsi.
In altre parole: si poteva individuare testando solo la modifica, oppure era un effetto collaterale difficile da anticipare?

Soluzione o miglioramento:

Indica come è stato risolto il problema o in che modo si sarebbe potuto evitarlo (ad esempio, mediante regression testing o test di integrazione più accurati).

ATTENZIONE: All'esame il Prof. Falessi specifica spesso che gli esempi non devono essere banali (come modifiche grafiche, di font o colore), ma devono riguardare **aspetti logici, funzionali o strutturali** del software.

L'esempio è il seguente esame di ispw:

Si descriva un esempio di **difetto** regressivo di tipo perfettivo, escludendo la modifica del font o simili.

Svolgimento.

- 1. Condizione iniziale:** Migliorare l'esperienza d'accesso.
- 2. Modifiche introdotte:** Aggiunta del metodo d'accesso tramite dati biometrici (impronta digitale o riconoscimento del volto).
- 3. Bug introdotto:** Alcuni dispositivi non hanno la possibilità di accedere a causa dell'assenza di dispositivi di lettura della biometria (lettore di impronta digitale o webcam per il riconoscimento del volto).
- 4. Prevedibilità:** Era prevedibile, considerando che il sistema è utilizzato da dispositivi eterogenei, come dispositivi mobili datati o PC desktop senza lettore di impronta o webcam.
- 5. Soluzione / Miglioramento:** Si potrebbe aggiungere un metodo di riconoscimento della mancanza dei dispositivi per la lettura dei dati biometrici ed in caso permettere l'accesso con le credenziali.

Bisogna inventare ed essere creativi.

1. Gli elementi visualizzati sono difficili da vedere perché troppo piccoli.
2. Si aumenta la dimensione degli elementi.
3. Alcuni elementi non sono più visibili perché tagliati fuori dalla finestra.

Questo è un tipico esempio di bug regressivo di tipo **perfettivo**, perché comunque il sistema funziona ma si cerca di migliorare la grafica. (All'esame questo non va bene).

Se al posto di 1 ci fosse stato scritto "Alcuni utenti non sono in grado di vedere affatto alcuni elementi perché troppo piccoli" sarebbe un esempio di modifica **correttiva**.

Il bug regressivo deve essere ragionevolmente creato applicando la modifica e non trovato testando la modifica, altrimenti non è un bug regressivo, per tutto quello che ci siamo detti.

In altre parole, il bug regressivo non è un errore "interna" alla modifica, ma un effetto collaterale indiretto che emerge dopo aver applicato il cambiamento. Se il problema viene riscontrato testando direttamente la modifica, allora **si tratta semplicemente di un bug nella modifica**, non di un bug regressivo.

- Il sistema è vittima di uno specifico tipo di attacco hacker.
- Si aumenta la complessità computazionale e di sicurezza dell'algoritmo di critto.
- Il sistema non funziona più per utenti che utilizzano degli HW vecchi.

Questo può essere **adattivo**, perché magari l'attacco hacker viene svolto da una nuova tecnologia sconosciuta, e quindi mi devo adattare, quindi ora che esiste mi devo adattare.

Potrebbe essere **correttivo** perché qualcuno si lamenta che i suoi dati vengano usati.

Può essere **perfettivo**, magari nessuno si è mai lamentato, questo attacco è sempre esistito e quindi io perfeziono la mia applicazione.

Potrebbe essere **preventivo** perché l'attacco hacker non è stato ancora usato.

Questo è un esempio di **modifica ibrida**.

- Il sistema è vittima di uno specifico tipo di attacco hacker ancora ¹mai usato.
- Si aumenta la complessità computazionale e di sicurezza dell'algoritmo di critto.
- Il sistema non funziona più per utenti che utilizzano degli HW vecchi.

Questo è di tipo perfettivo/preventivo.

- Il sistema è vittima di uno specifico tipo di attacco hacker in base ¹ad una tecnologia adesso in voga.
- Si aumenta la complessità computazionale e di sicurezza dell'algoritmo di critto.
- Il sistema non funziona più per utenti che utilizzano degli HW vecchi.

Questo è adattivo.

- Il sistema è vittima di uno specifico tipo di attacco hacker in base ¹alle lamentele clienti.
- Si aumenta la complessità computazionale e di sicurezza dell'algoritmo di critto.
- Il sistema non funziona più per utenti che utilizzano degli HW vecchi.

Questo è correttivo.

All'esame verrà chiesto un esempio di bug regressivo per ogni tipologia di modifica.

Questi sono classici esercizi d'esame.

(Se esce il bug regressivo)

I cinque punti illustrati adottano una notazione differente rispetto a quella utilizzata dal professor Falessi, pur mantenendosi pienamente coerenti con la sua impostazione teorica.

L'esempio precedentemente riportato, relativo ai dati biometrici, è stato valutato positivamente dallo stesso Professore, senza alcuna correzione.

Conclusione:

Il bug regressivo è in realtà un concetto molto semplice: occorre soltanto immaginare un errore che si verifica a seguito di una modifica apportata al software, in una parte del sistema che prima funzionava correttamente. Non è quindi necessario complicare l'esempio con casi estremi o teorici: basta pensare a una modifica ragionevole e a un effetto collaterale imprevisto che ne derivi.

FINE.